R

Rumbo

CRM para agencias de turismo PyME

Punto de Control 1 · Análisis, diseño y planificación

Francisco Luciano · Nahuel Viniegra · Paloma Patiño

Universidad de Morón · Ingeniería Informática · Trabajo de Campo · 02/10/26

AGENDA

Qué vamos a mostrar

1. **Definición del problema**
   Cómo trabaja hoy la agencia y qué ofrece el mercado
2. **Alcance y límites**
   Objetivos, alcance, actores y casos de uso
3. **Modelo conceptual**
   Frontend, backend y base de datos; interfaces
4. **Modelo de flujo**
   Diagramas UML y modelo C4
5. **Modelo de datos SQL**
   Entidad-relación y script
6. **Comunicación V–C**
   MVC y contrato OpenAPI en JSON
7. **Modelos y agentes de IA**
   El modelo del producto, los agentes y sus prompts
8. **Modelado con Mermaid.js**
   Diagramas como código

Cómo trabaja hoy la agencia

1. **Carpetas físicas por viaje**

   No hay un lugar único para ver clientes y viajes juntos.

2. **Consultas por Instagram, WhatsApp o en persona**

   El seguimiento depende de la memoria de quien atendió.

3. **Cobros y vencimientos en planillas**

   Señas, cuotas y pagos a operadores, sin avisos automáticos.

4. **Vendedores freelance y operaciones en ARS y USD**

   Hay que saber qué vendió cada uno y cuánto ganó la agencia.

Consecuencias

Se pierden ventas

Se olvidan cobros

No queda memoria del cliente

Y el negocio se sostiene con clientes fieles: llegan pocos nuevos.

Por qué no alcanzan las herramientas que existen

Necesidad de la agencia	Excel y mensajería	CRM genérico	CRM turismo EE. UU.	Rumbo
Seguir la consulta hasta la venta	A mano	Sí	Sí	**Sí**
Viaje con itinerario, operador y economía	A mano	No	Sí	**Sí**
Cuotas y vencimientos con avisos	Sin avisos	No	Parcial	**Sí**
Ganancia y comisión freelance en ARS o USD	A mano	No	Parcial, USD	**Sí**
Recordatorios por requisitos del destino	No	No	No se encontró	**Sí**
Recontactar clientes fieles	No	A medida	No se encontró	**Sí**
Español, pesos y costo PyME	Sí	Parcial	No	**Sí**

Fuente: documentación pública de HubSpot, Pipedrive, Tern, TravelJoy, Travefy y SAVIA, consultada el 30/09/26. «No se encontró» = la documentación no lo menciona.

Qué busca resolver Rumbo

1

Digitalizar y centralizar

De las carpetas a un único sistema de clientes y viajes.

2

Fidelizar clientes

Ficha con preferencias, historial y recontacto.

3

Ordenar el circuito de venta

De la consulta a la cotización y el cierre, sin perder pasos.

4

Controlar la economía

Ganancia, margen, comisión y cuotas, en ARS o USD.

5

No perder vencimientos

Alertas de pagos, formularios por destino e insistencia.

6

Decidir con datos

Dashboard, estadísticas y reportes mensuales.

Alcance y límites

11 módulos · web y mobile

- **Clientes**
- **Oportunidades y cotización**
- **Viajes**
- **Economía y pagos**
- **Alertas y recordatorios**
- **Catálogo de hoteles**
- **Redes sociales**
- **Dashboard y reportes**
- **Calendario de viajes**
- **Configuración y catálogos**
- **Usuarios y seguridad**

Fuera de esta etapa

- Pagos online, reembolsos y penalidades
- Publicación automática en redes
- Integración con mayoristas y GDS
- Cotizador y ofertas por scraping
- Avisos automáticos al pasajero
- Facturación (fase 2)

Incluido: CRM proactivo (recontacto de clientes fieles) y vencimiento de pago al proveedor por hotel.

Actores y casos de uso

Empleado

Usuario principal

Opera todo el circuito: clientes, consultas, cotizaciones, viajes, cobros y alertas.

Admin

Hereda del empleado

Configura comisiones, destinos, operadores y usuarios, y modera reseñas.

Dueña

Hereda del empleado

Mira el negocio: dashboard, reportes y alertas de toda la empresa.

Freelance

Hereda del empleado

Vendedor externo: ve y opera solo lo suyo; no toca el catálogo ni las redes.

Pasajero

Actor externo

Consulta, recibe cotizaciones y paga. No opera el sistema.

31

casos de uso en 11 módulos

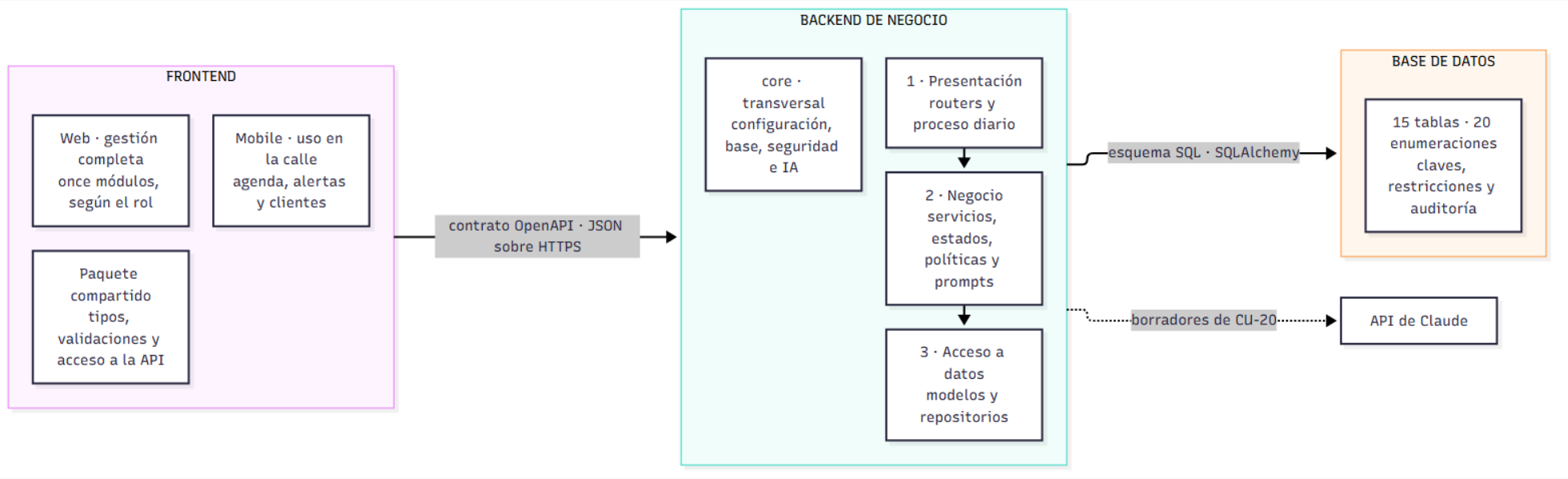
155

requisitos funcionales

16

requisitos no funcionales

Tres capas: frontend, backend y base de datos



Frontend

Web con los once módulos según el rol, y mobile con agenda, alertas y clientes. Muestra y valida; no decide reglas.

Backend de negocio

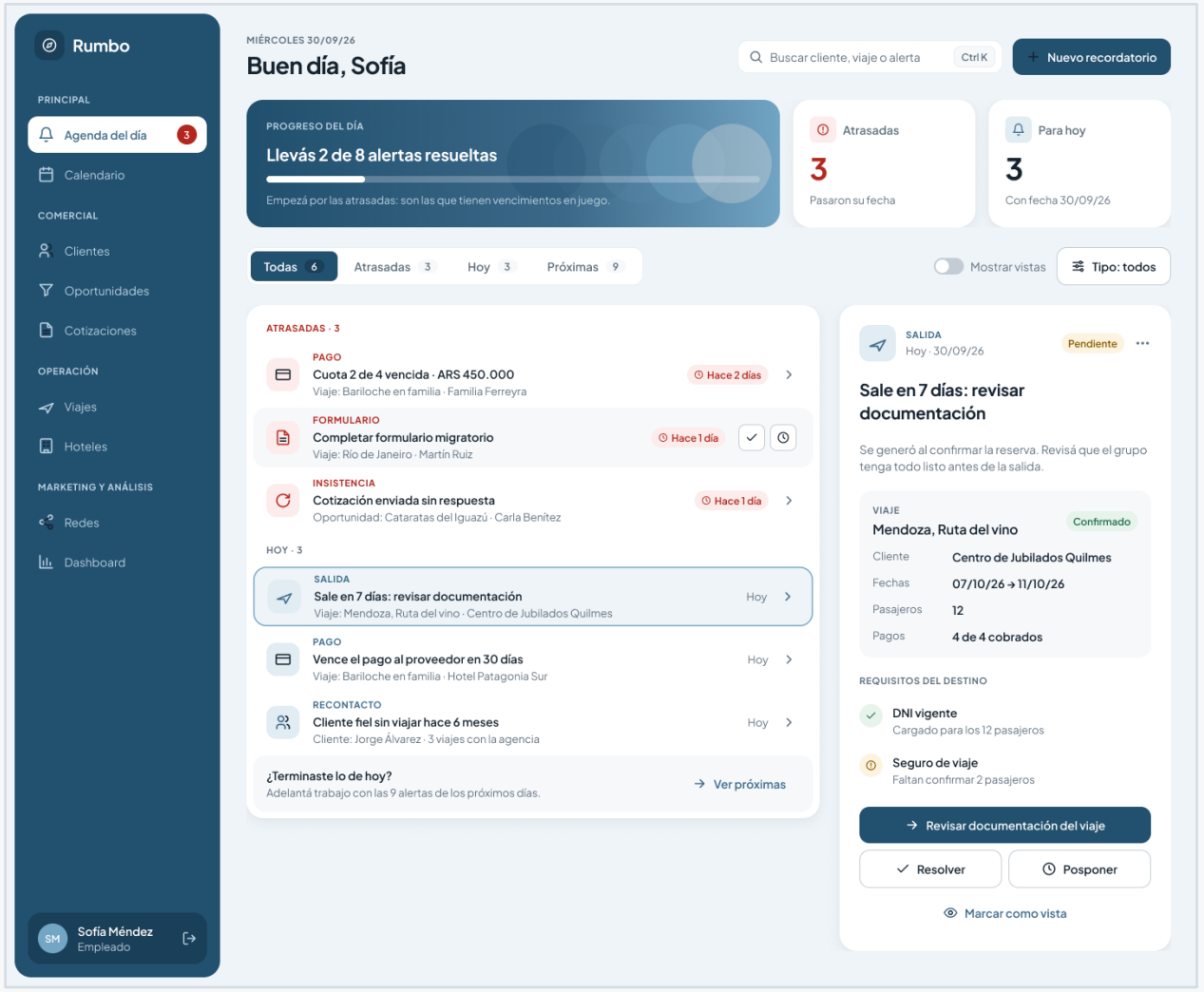
La única puerta al dominio: permisos por rol, un caso de uso por transacción, estados y el proceso diario.

Base de datos

PostgreSQL con 15 tablas: protege la integridad con claves y restricciones; todo es baja lógica.

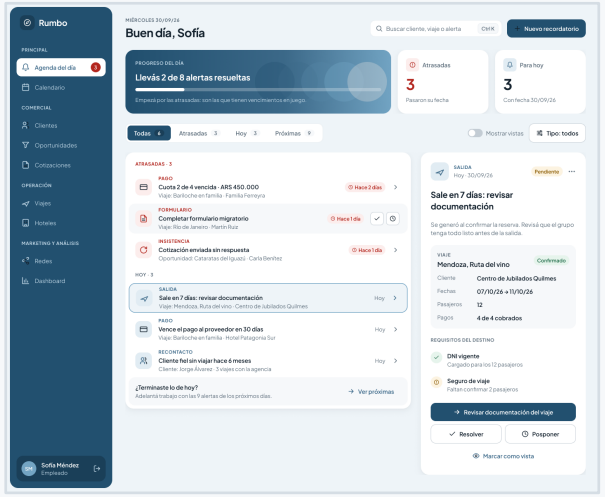
Interfaces gráficas: prototipo en Figma

1. Validar el estilo con una pantalla: la agenda del día
2. Sistema de diseño liviano: paleta del equipo, Plus Jakarta Sans, 7 componentes y 28 íconos
3. Pantallas clave del circuito y después el resto de los casos de uso
4. Vistas por perfil y mobile
5. Navegable: cada pantalla se alcanza desde un punto de inicio



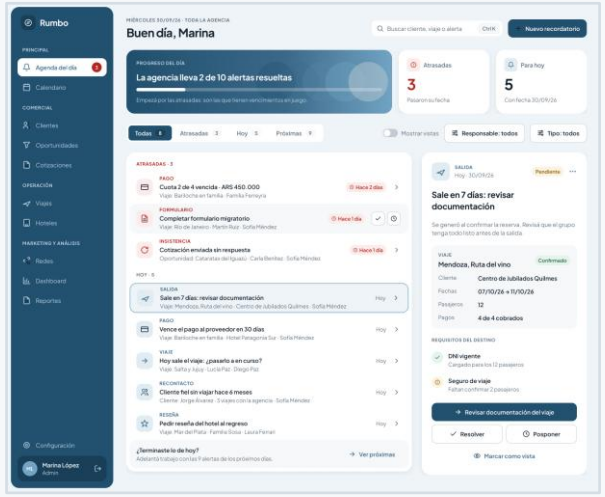
Contraste AA · el color nunca es la única señal · «Deshacer» en lugar de confirmar

Una vista por perfil



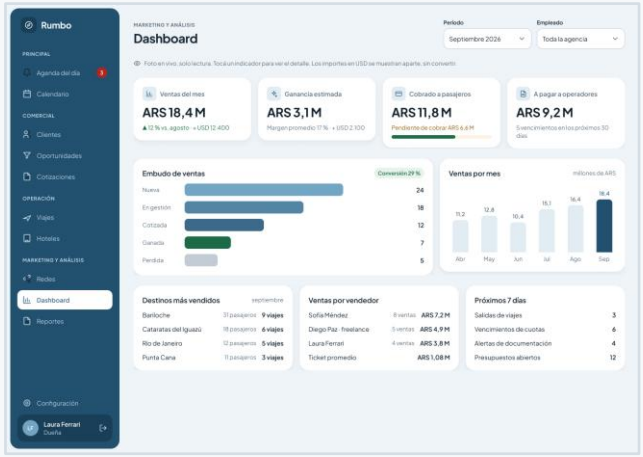
Empleado

Todo el circuito; su agenda y su dashboard



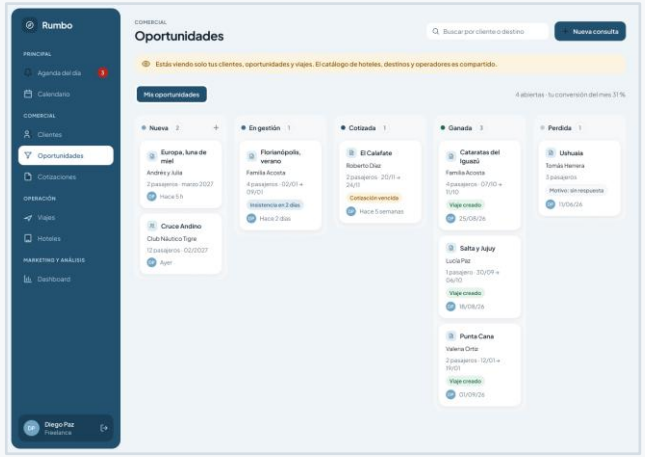
Admin

Alertas de toda la agencia y configuración



Dueña

Todo lo del admin, más el dashboard y los reportes de la empresa

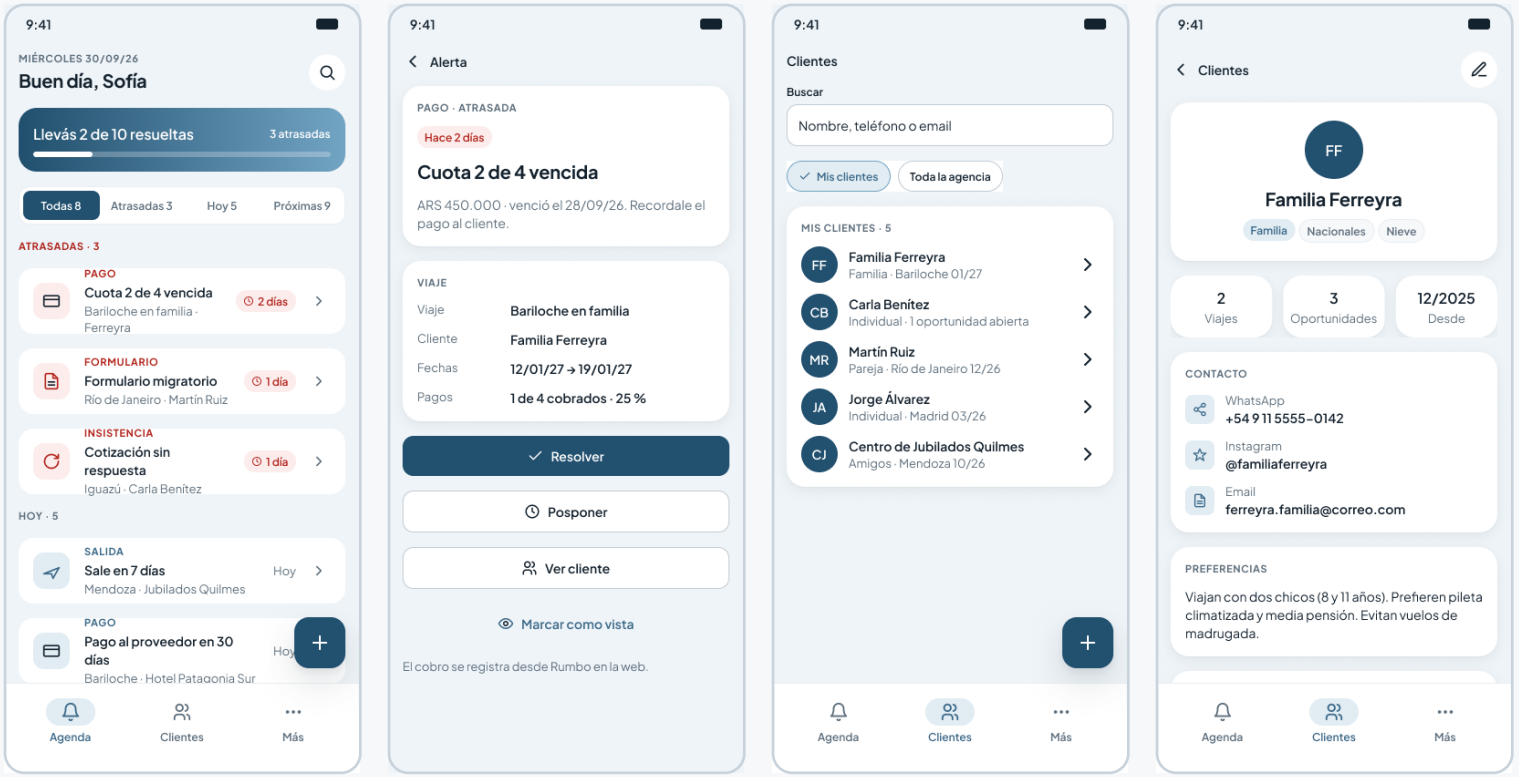


Freelance

Solo sus clientes, oportunidades y viajes

Mobile: agenda, alertas y clientes

- Ver la agenda y atender alertas: resolver, posponer, marcar como vista
- Crear recordatorios
- Buscar clientes, ver su ficha y dar de alta uno nuevo
- Cotizar, gestionar viajes y cobrar: desde la web



React Native + Expo · la misma API y el mismo contrato que la web

Recorrido del prototipo

59

pantallas

5

puntos de inicio

1. Rumbo · escritorio (empleado): inicio de sesión → agenda → cliente → consulta → cotización → viaje → cobro
2. Rumbo · mobile
3. Dueña · dashboard
4. Admin · configuración
5. Freelance · embudo

figma.com/design/5rq8HOlZQ3uu4IUpBuhuZp · Present → Opciones → «Ajustar»

El circuito que Rumbo digitaliza

1

Cliente

Se registra o se busca

2

Consulta

Nace la oportunidad

3

Cotización

Versionable, con vigencia

4

Reserva y seguimiento

Se crea el viaje

5

Cobros y alertas

Cuotas, vencimientos y post-viaje

Estados del viaje

reservado → **vendido** → **confirmado** → **en curso** → **finalizado**

El primer cobro pasa el viaje a **vendido**; si salda el total, a **confirmado**. Un proceso automático corre todos los días a las 00:00: vence pagos y cotizaciones y genera alertas.

Modelo de dominio: diagrama de clases

13

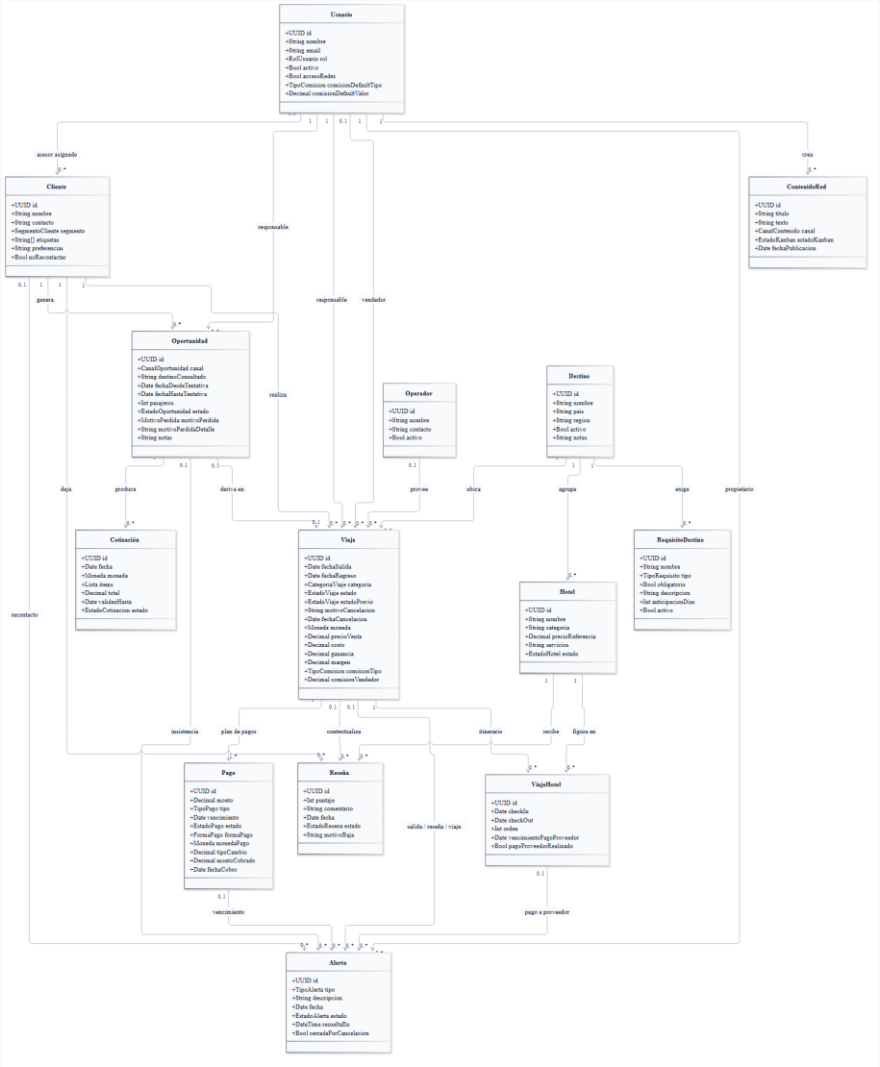
entidades del dominio

20

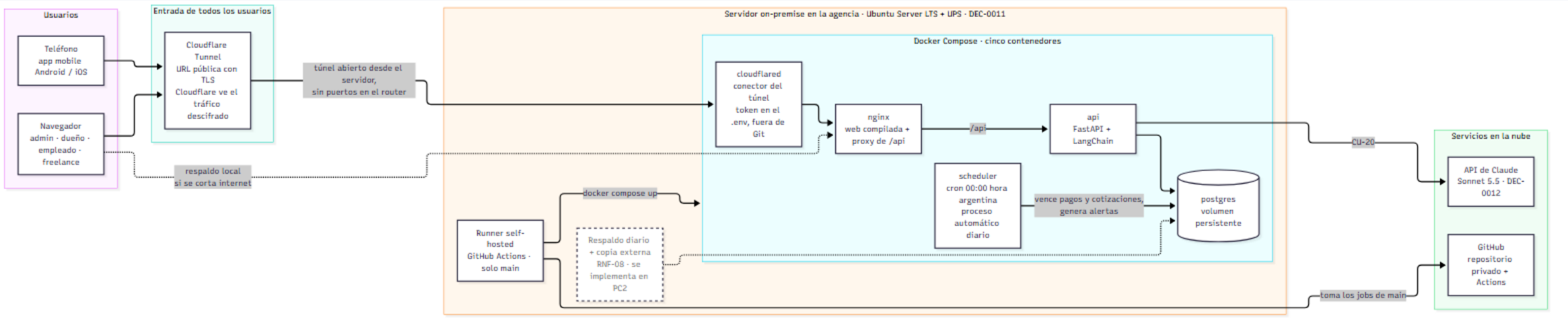
enumeraciones

6

máquinas de estado



Arquitectura: vista general y despliegue

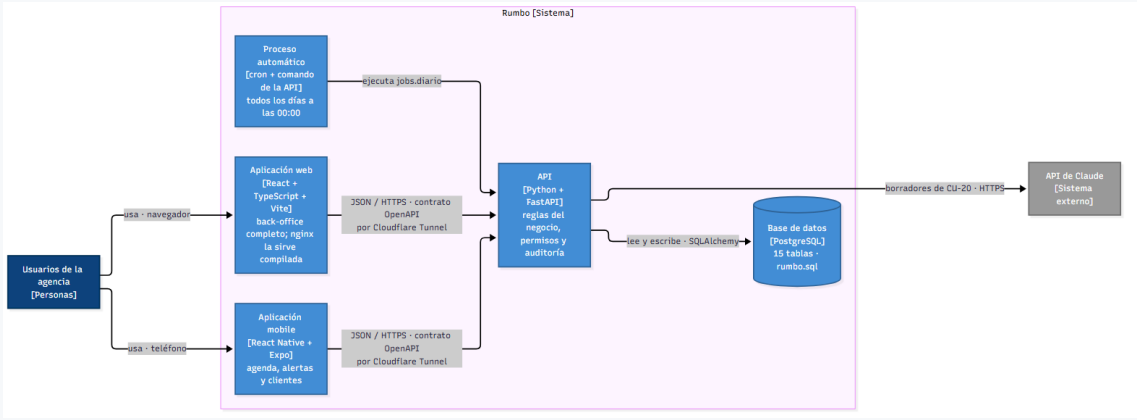


- Cliente-servidor: web y mobile hablan en JSON con una única API
- FastAPI + PostgreSQL + LangChain
- Servidor on-premise en la agencia con Docker Compose
- Proceso automático diario a las 00:00
- Acceso externo por Cloudflare Tunnel
- Respaldo diario con copia fuera del servidor

Modelo C4: contexto y contenedores

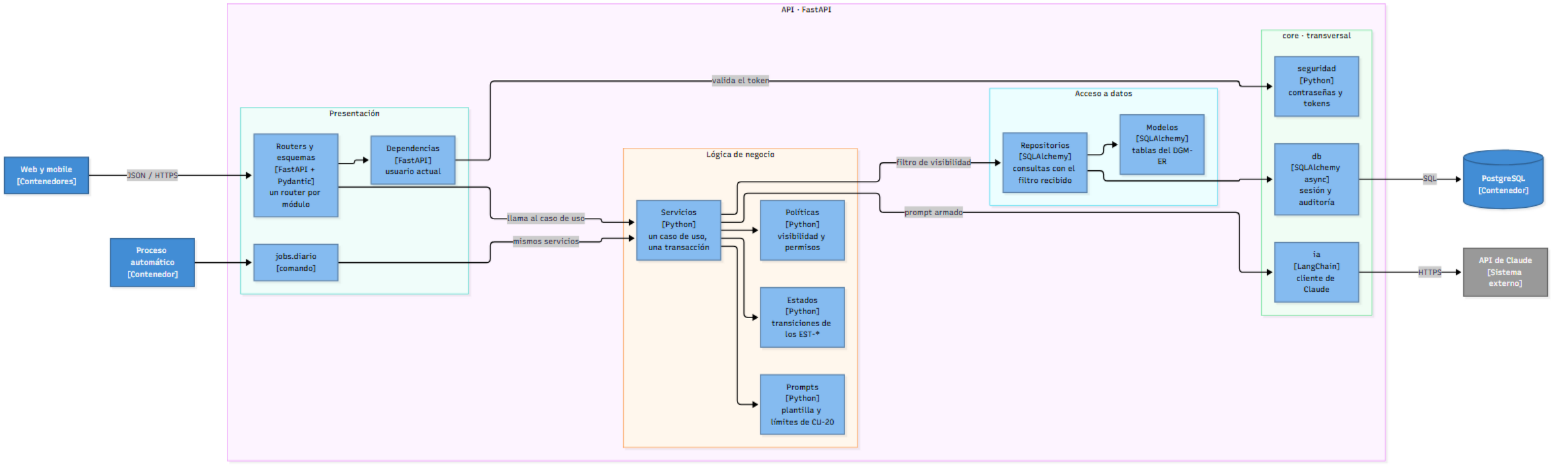


Nivel 1 · Contexto



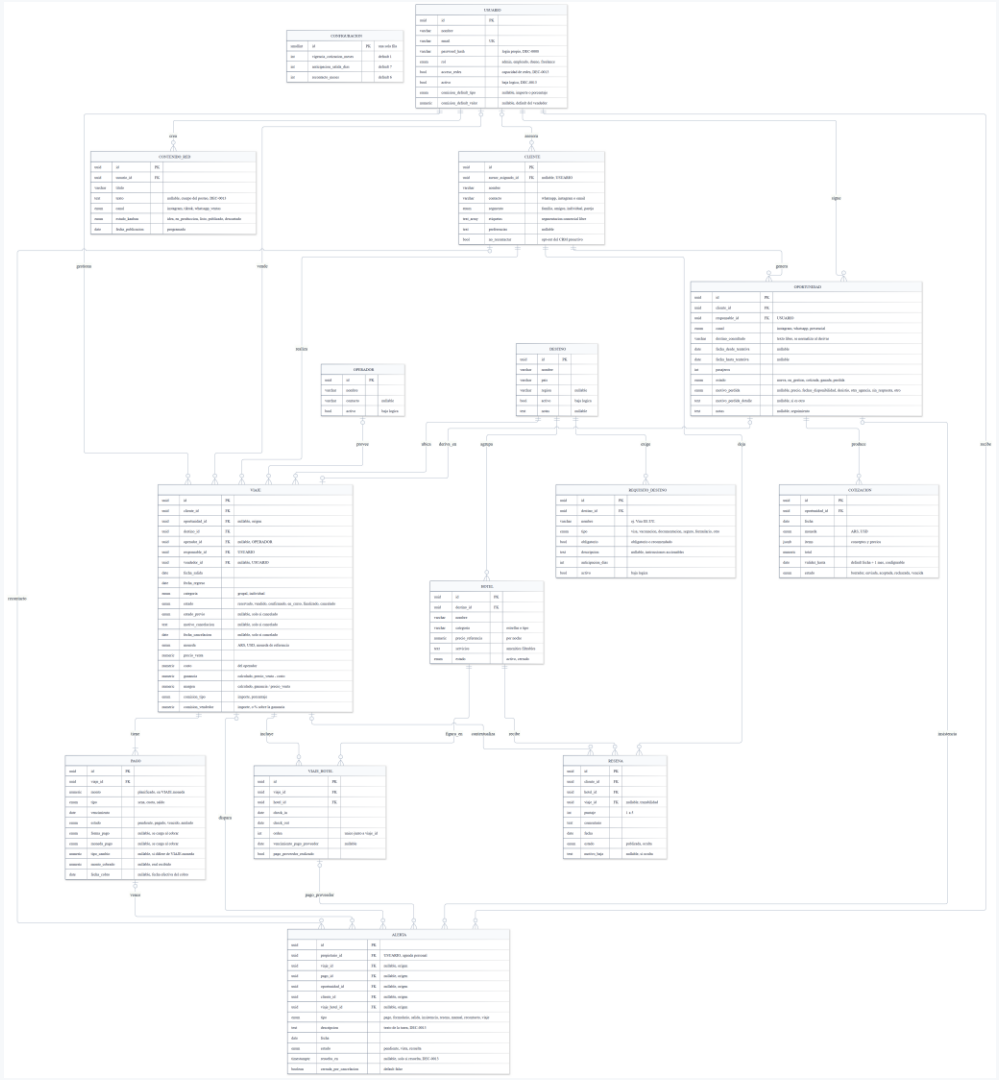
Nivel 2 · Contenedores

Modelo C4: componentes de la API



- Tres capas: presentación, negocio y datos, más core
- Un paquete por módulo funcional
- Visibilidad por fila: el filtro por rol es obligatorio en toda lectura
- Auditoría de autor y fecha en cada cambio

Modelo entidad-relación y SQL

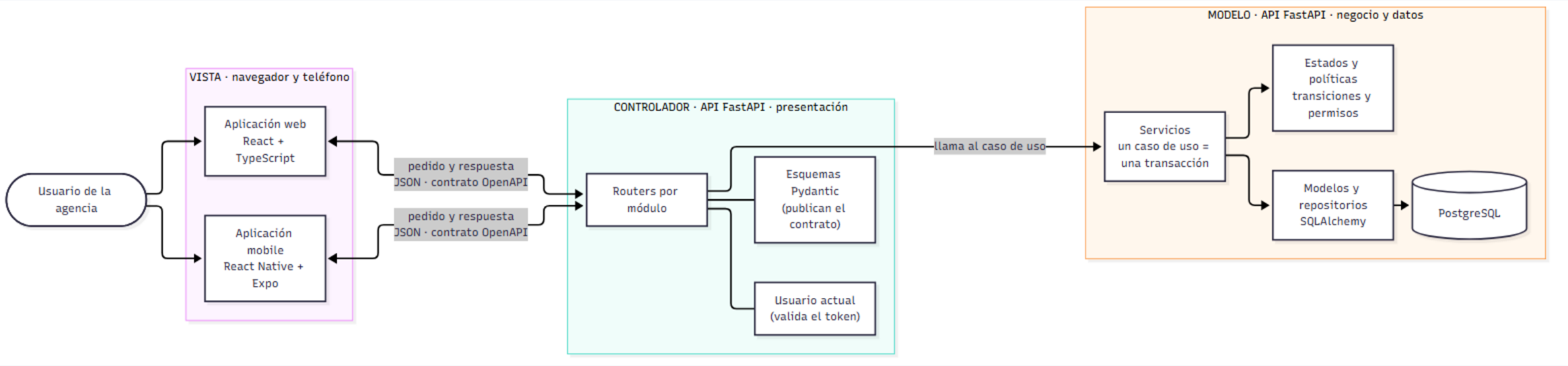


Script SQL · PostgreSQL

- 15 tablas y 20 tipos enumerados nativos
- Claves UUID con gen_random_uuid()
- Auditoría en todas las tablas: quién creó y modificó cada registro
- Ganancia y margen como columnas calculadas
- CHECK: cada alerta tiene a lo sumo un origen
- Validado corriéndolo en PostgreSQL 18 con casos de prueba permitidos y rechazados

En PC2, la primera migración de Alembic lo reproduce.

MVC en Rumbo



Vista

Web (React) y mobile (React Native)

Controlador

Routers y esquemas de la API FastAPI

Modelo

Servicios, estados, políticas y PostgreSQL

Vista y Controlador se comunican en JSON según el contrato OpenAPI.

Comunicación Vista–Controlador: OpenAPI

96
operaciones REST

11
módulos

4
roles con permisos por operación

- OpenAPI 3.1 en JSON, validado con Redocly
- Cada operación declara el rol que la puede usar y los casos de uso que realiza
- Errores uniformes en formato application/problem+json
- Esquemas completos del circuito central: Cliente, Oportunidad, Cotización, Viaje, Pago y Alerta
- Web y mobile generan sus tipos desde el contrato (openapi-typescript)

La IA dentro del producto

Claude Sonnet 5.5

Borradores de publicaciones para redes (CU-20)

- Entrada: tema, canal y tono
- Un borrador por pedido, que se puede regenerar
- Integración con LangChain desde el backend
- No recibe datos de clientes
- No publica: la publicación es manual

Por qué este modelo

	Sonnet 5.5	Haiku 4.5
Calidad del texto en español	**Mejor**	Menor
Entrada (USD por millón de tokens)	**2**	1
Salida (USD por millón de tokens)	**10**	5
Elegido	**Sí**	No

Con unas decenas de borradores por mes, la diferencia de costo es de centavos de dólar. Lo que percibe la agencia es la calidad del texto.

Cómo trabajamos con agentes de IA

Claude Code

Agente principal: lee el vault, edita notas y diagramas, genera entregables y hace commits.

Subagentes

Exploran el vault en paralelo cuando la búsqueda es amplia.

MCP de Figma

Construye el prototipo de interfaces directamente en Figma.

El repositorio es la memoria compartida: lo que no queda escrito no pasa de una sesión a otra.

1

Entrar en contexto

Trae lo nuevo del repositorio y lee el estado y la bitácora

2

Trabajar

Edita solo las notas de la tarea y respeta las decisiones

3

Verificar

Diagramas, entregables y enlaces

4

Registrar y subir

Bitácora, commit con la identidad de cada uno y push

Los prompts con que construimos Rumbo

1 · Prompt de sistema

`CLAUDE.md`

- El agente lo lee al empezar cada sesión.
- Cómo entrar en contexto: identificar a la persona, traer el trabajo del equipo y leer el estado.
- Reglas: respetar el alcance, no inventar datos del negocio y no aprobar nada sin el equipo.
- Cómo cerrar: bitácora, commit con la identidad de la persona y subir.

2 · Prompt de cada sesión

«Hola, entrá en contexto.»

- Con CLAUDE.md alcanza un pedido corto: el agente aplica el protocolo solo.
- Después, cada pedido describe la tarea: «revisemos y decime qué falta».
- Fuera de Claude Code, un prompt inicial le indica qué leer y qué no formalizar.

3 · Prompts reutilizables

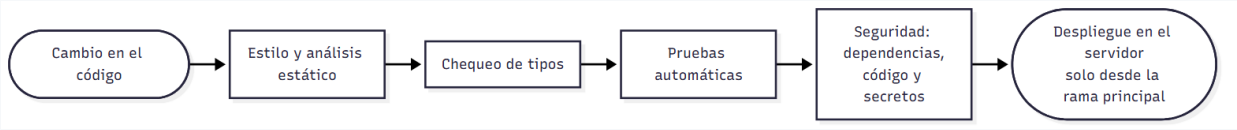
`Skills del proyecto`

- Un procedimiento que se repite se escribe una vez como skill, versionada en el repositorio.
- Ejemplo: gestionar-skills, para incorporar una skill nueva con su origen, versión y licencia.
- Mismo pedido, mismo resultado, para cualquier integrante.

Diagramas como código

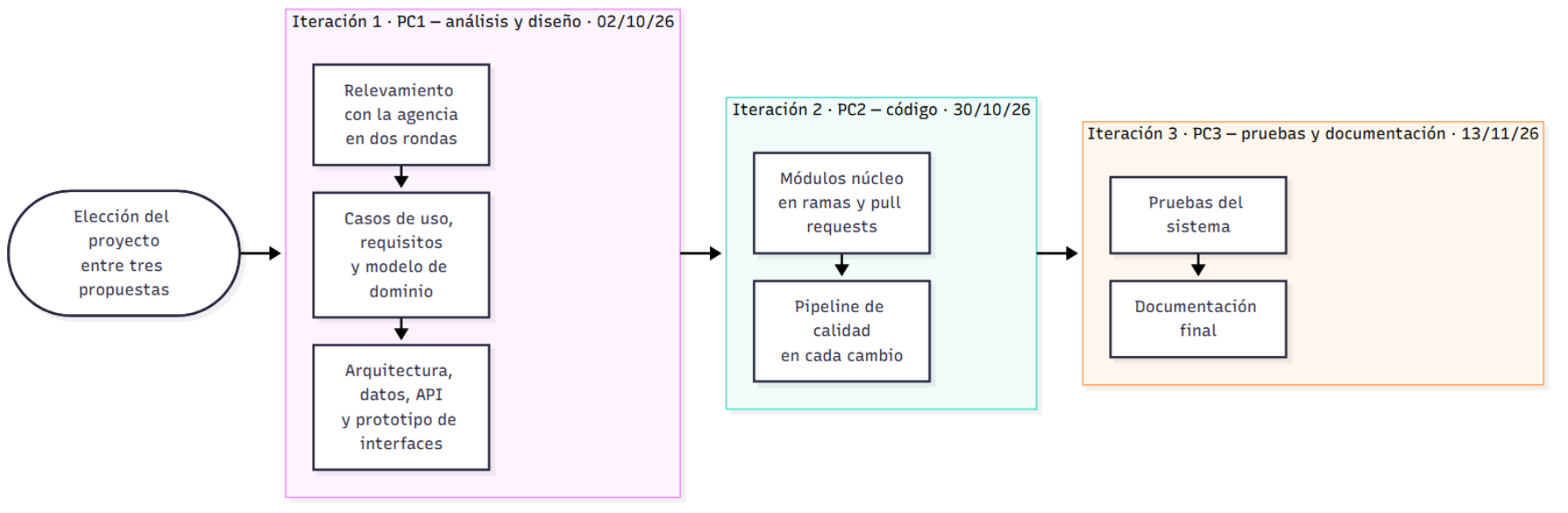
- Los 34 diagramas del proyecto están escritos en Mermaid.js, dentro de la nota que los explica.
- Se versionan en Git: cada cambio queda registrado con su autor.
- La imagen se genera desde el código, nunca a mano, con mermaid-cli y el motor ELK.
- Cubre clases, secuencia, estados y entidad-relación; casos de uso, actividad y C4 con diagramas de flujo.

```
flowchart LR
  Push(["Cambio en el código"]) --> Lint["Estilo y análisis estático"]
  Lint --> Tipos["Chequeo de tipos"]
  Tipos --> Tests["Pruebas automáticas"]
  Tests --> Seg["Seguridad: dependencias,<br/>código y secretos"]
  Seg --> Deploy(["Despliegue en el servidor<br/>solo desde la rama principal"])
```



Este es el código completo del diagrama del pipeline de calidad, y la imagen que se genera a partir de él.

Modelo de desarrollo: iterativo e incremental



Una sola fuente de verdad

Todo en un repositorio Git, en Markdown y Mermaid.

Borrador → revisión → formalizado

Nada se formaliza sin la aprobación de los tres.

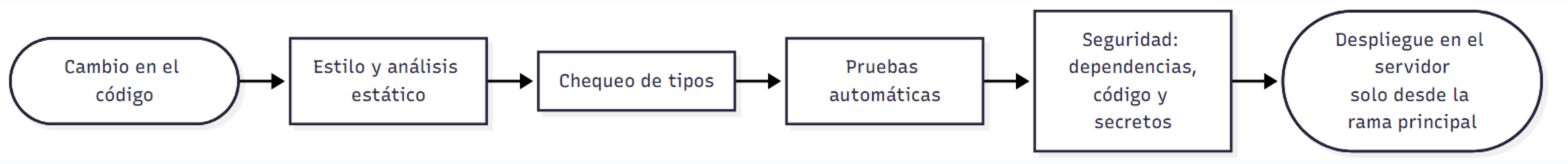
Decisiones registradas

13 decisiones con alternativas y justificación.

Sync y frentes

Sync semanal; cada uno lleva un frente y el resto revisa.

Git y control de calidad



Git y trazabilidad

- Documentación: commits chicos sobre la rama principal
- Código (desde PC2): una rama por funcionalidad y pull request revisado
- Cada commit cita la regla o decisión que implementa
- Cada integrante con su propia identidad de Git

Parte	Estilo y tipos	Pruebas
Backend	ruff, mypy	pytest
Web	ESLint, TypeScript	Vitest, Playwright
Mobile	ESLint, TypeScript	Jest, Maestro

Seguridad: pip-audit, pnpm audit, bandit, Semgrep y gitleaks. Dependabot propone las actualizaciones.

CIERRE

Resumen del PC1

31
casos de uso

155
requisitos funcionales

13
entidades

13
decisiones registradas

59
pantallas prototipadas

96
operaciones de API

Próximo: PC2 · 30/10/26 — código funcional de los módulos núcleo, sobre este diseño, en ramas y pull requests.